Phone Automation Platform — SRS (Flow 1: TikTok draft publish)

Oct 23, 2026 · @Malik

Verdict: the flow is buildable and will run on Android 15, with four conditions

Yes, build it. Every step (catch the TikTok notification, open the draft, tap Next, tap Post) uses two standard, non-root Android APIs: `NotificationListenerService` and `AccessibilityService`. The same APIs power Tasker, MacroDroid and the open-source phone agents. No platform wall blocks this on Android 15, and TikTok's post screen is driven by other automation tools daily.

The flow succeeds when these four conditions hold. Everything else in this document is detail on how to meet them or what happens when they fail.

#	Condition	Why it matters	How we meet it
1	Phone is unlocked (or swipe-lock only) and online when the notification arrives	Accessibility clicks do not work on the lock screen; uploads need network	Bot wakes the screen; phone runs with swipe lock or Extend Unlock; see Prerequisites
2	Our app is installed via `adb install` and both services are enabled	Android 13+ blocks accessibility for APKs opened from a file manager	Install from Android Studio or ADB; a watchdog warns if a service is off
3	TikTok is logged into the account OnePublish authorized, in English, up to date	The inbox draft belongs to one account; selectors match English button text	One-time setup; bot aborts on unexpected screens
4	TikTok's editor and post screens still show text buttons "Next" and "Post"	Text matching is the stable selector	Selectors live in an editable config; a dump script re-checks them after TikTok updates

Two things cannot be settled from research alone and need a one-hour test on your phone before writing the app (section 10): whether `contentIntent.send()` opens the draft from the background on your OEM build (it should, with the fallback of tapping the

notification in the shade), and the exact node attributes of "Next" and "Post" on your TikTok version.

What you are not getting: captions typed by the bot (OnePublish can pre-fill them through TikTok's API), handling of locked phones with a PIN you keep, or survival across TikTok redesigns without a config update.

Scope of v1

The app is a personal automation platform for the phone: a shared engine that runs flows, each with its own one-tap switch. v1 ships the first flow, TikTok draft publish, hardcoded; other flows are added on the same engine afterwards.

In scope

- Trigger: a TikTok notification whose title matches "Your content from OnePublish is ready" and whose text matches "Edit your video before sharing to TikTok".
- Action: open that notification, wait for the editor, tap **Next**, wait for the post page, tap **Post**, confirm the upload started, return home.
- Enable/disable this flow with one tap: a Quick Settings tile, plus the same switch inside the app (a master switch covers all flows later). Off means the flow ignores its trigger; services stay granted.
- A run log you can read: when it fired, each step's result, a screenshot on failure, and a notification from the bot on success or failure.
- A watchdog that catches a matching notification the listener missed (phone rebooted, process killed) and warns when either service is disabled.
- A dry-run mode that walks the flow and logs what it would click, without clicking Post.

Out of scope for v1 (planned as later rules)

- Other flows (WhatsApp, Discord, Facebook, alarms). They follow v1 as new Flow classes on the same engine; v1 only has to keep that door open (NFR-8).
- Editing the caption, cover, hashtags, audience or any post setting; the bot takes what TikTok shows.
- Entering a PIN or pattern to unlock the phone.
- A visual rule builder, DSL or recorder; v1 flows are Kotlin classes.
- Any LLM or vision model; v1 is deterministic text matching.
- Publishing the app anywhere; it is sideloaded on your Android 15 phone only.

How the flow works today

OnePublish uses TikTok's Content Posting API in **Upload** (inbox) mode, which is why the video needs a manual finish. TikTok documents this mode as: the app uploads the video, TikTok sends a notification to the creator's inbox, and the creator completes the post through TikTok's own editing flow (Content Posting API overview, Upload reference). The notification wording you see matches TikTok's documented template, "Your video from <APP> is ready: Edit your video before sharing to TikTok" (legacy Share Video doc).

What happens end to end:

1. OnePublish calls `/v2/post/publish/inbox/video/init/` and uploads the file. TikTok processes it server-side.
2. TikTok's status becomes `SEND_TO_USER_INBOX` and TikTok pushes a notification to your phone. It also appears inside TikTok under Inbox → System notifications, so a missed push is recoverable in-app.
3. Tapping the notification opens TikTok's editor with the video preview. **Next** leads to the post page (caption, audience, settings). **Post** uploads it.
4. TikTok reports `PUBLISH_COMPLETE` to OnePublish once the post is made; OnePublish may show that status.

Facts from TikTok's docs that shape the design:

Fact	Source	Consequence for the bot
TikTok caps pending inbox shares at 5 per rolling 24 hours per creator	Upload reference	If the bot is off or failing, OnePublish uploads start failing after 5 unposted drafts. Post promptly; alert on failures
Title and description can be sent with the upload and are pre-filled in the editing flow	Photo/content init reference	The bot never types a caption. Set it in OnePublish
Inbox upload needs TikTok app version ≥ 31.8	same	Keep TikTok updated
Unaudited API clients get private-only posts	Content sharing guidelines	Audience is decided by OnePublish's audit status and the draft's settings, not by the bot

Fact	Source	Consequence for the bot
Third-party tools report inbox drafts expire if not posted within ~24 hours	Postiz docs (not in TikTok's docs; treat as unconfirmed)	Another reason to post within minutes of the notification

The exact screens on your phone are the one thing docs cannot give: TikTok ships UI changes weekly. The verification spike (section 10) records them.

Android 15: what the platform allows and blocks

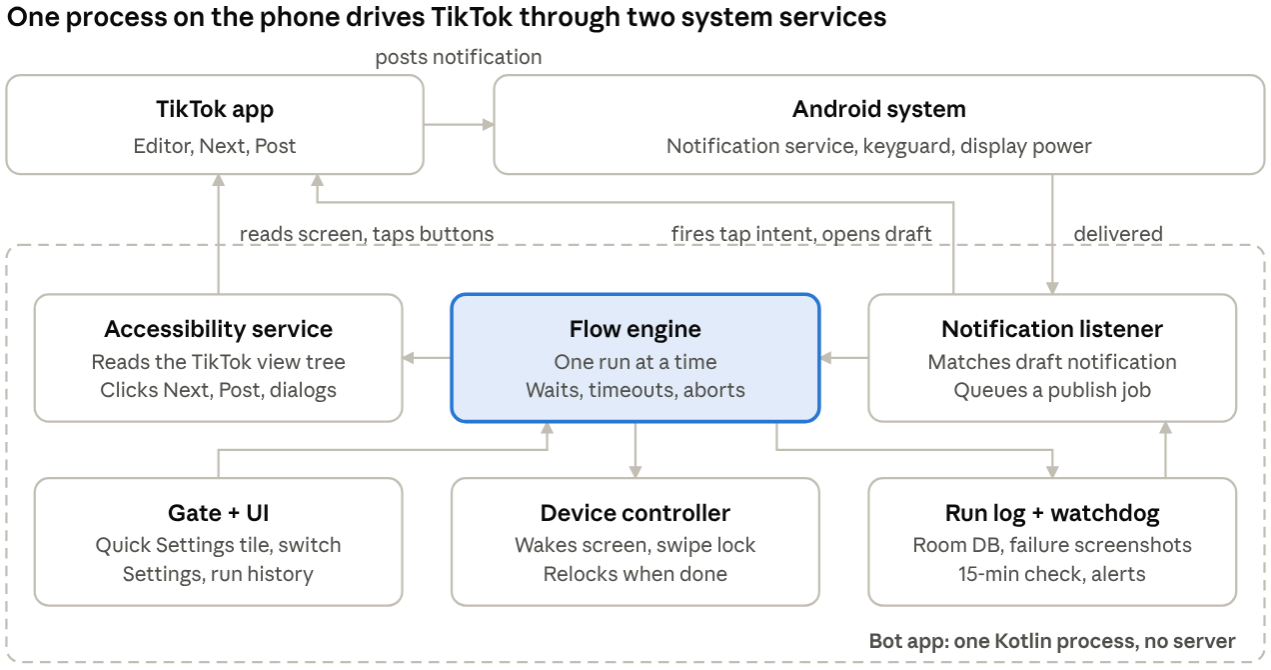
Every capability the bot needs exists on Android 15 without root. The table lists each one, the rule that governs it, and the design choice that follows.

Capability	API	Android 15 rule	Design choice
Receive the TikTok notification with its title, text and tap action	`NotificationListenerService`	User grants "Notification access" once. Android 15 hides only OTP-type notifications from untrusted listeners (Android Authority); TikTok's is not one	Listener filters on package + title + text; group-summary notifications are ignored
Open the draft without touching the screen	`notification.contentIntent.send()`	A background app may start another app's activity only under a BAL exception. Apps with a **system-bound AccessibilityService** are on the exception list (background starts). Targeting API 34+, the sender must opt in with `setPendingIntentBackgroundActivityStartMode(MODE_BACKGROUND_ACTIVITY_START_ALLOWED)`; a launch is allowed when at least one of creator or sender both opts in and qualifies for an exception (secure BAL)	Send with the opt-in while our accessibility service is bound. A blocked launch throws nothing and only logs "Background activity launch blocked!", so the bot verifies TikTok came to the front within 8 s, else falls back to opening the shade and tapping the notification node
See TikTok's screen and press its buttons	`AccessibilityService`	User enables it once under Settings → Accessibility. Reads the view tree, clicks nodes, injects gestures, presses Back/Home, opens the notification shade, locks the screen (`GLOBAL_ACTION_LOCK_SCREEN`), takes screenshots (`takeScreenshot`, API 30+)	Service config: `canRetrieveWindowContent`, `canPerformGestures`, `canTakeScreenshot`, `packageNames` = TikTok + `com.android.systemui`, event types for window state/content changes
Enable both services for a sideloaded app	Restricted settings (Android 13+)	Blocked for APKs installed through a file manager or browser; allowed for `adb install` and for store-style session installs (XDA)	Install from Android Studio/ADB. If ever blocked: App info → ⋮ → Allow restricted settings, or `adb shell appops set <pkg> ACCESS_RESTRICTED_SETTINGS allow`

Capability	API	Android 15 rule	Design choice
Stay alive for days	Bound services + optional foreground service	Both services are bound by the system and restarted if the process dies. OEM battery managers may still kill the process	Request battery-optimization exemption; foreground service of type `specialUse` as belt-and-braces; OEM autostart setting documented in Prerequisites
Wake the screen	Transparent activity with `setTurnScreenOn` + `setShowWhenLocked`, or a wake lock	Allowed from a BAL-exempt app	Used only when the display is off; the bot restores the lock afterwards
Dismiss the lock screen	`KeyguardManager.requestDismissKeyguard`	Works only for a non-secure (swipe) keyguard, or when Extend Unlock already treats the device as unlocked	Phone must be effectively unlocked at trigger time; PIN entry is out of scope
Run after reboot	`BOOT_COMPLETED` receiver	Android 15 blocks some foreground-service types from boot; `specialUse` is allowed	Services are re-bound by the system anyway; the receiver only restarts the watchdog

Two rules that do **not** apply to this phone yet but will matter later: Android 16's Advanced Protection Mode and Android 17's version of it, which revokes accessibility from apps not declaring `isAccessibilityTool="true"` (Android Authority). Keep that mode off after upgrading, or declare the flag.

System architecture



architecture: 2 system services, 1 flow engine, no backend

The notification listener is the trigger, the accessibility service is the hands, and the flow engine between them owns every wait and timeout. Nothing leaves the phone.

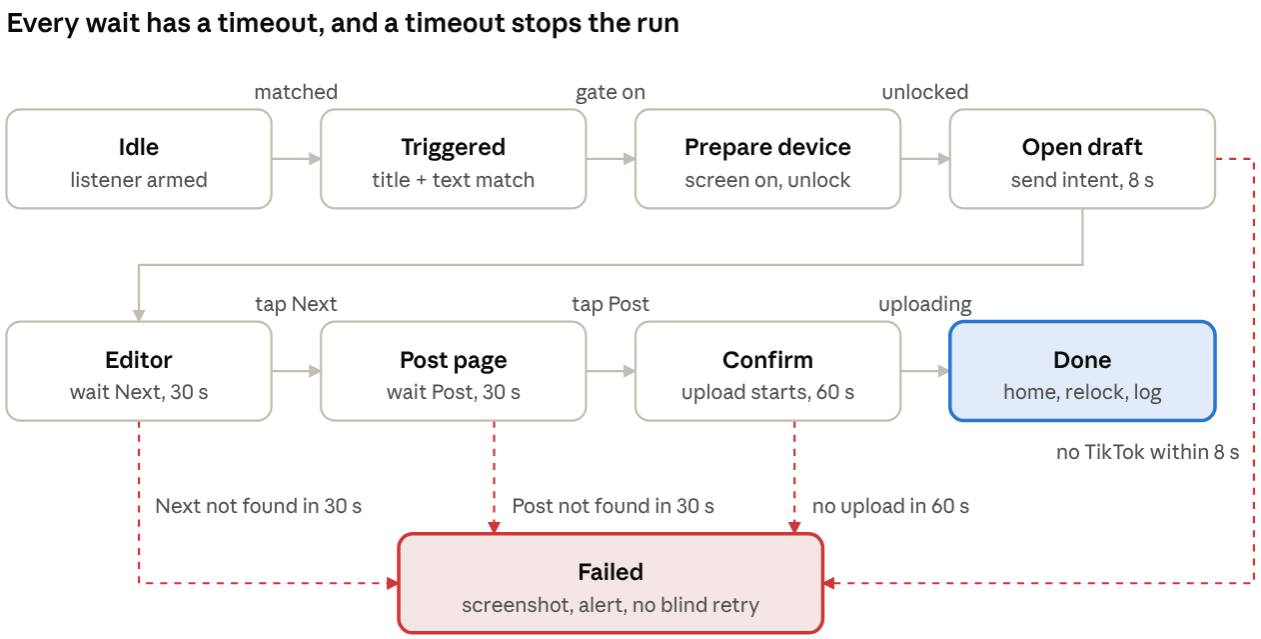
Stack: Kotlin, Jetpack Compose, minSdk 35 = targetSdk 35, one Gradle module, no backend, no network permission. Selectors and timeouts live in `selectors.json` in the app's files directory, so a TikTok UI change is a config push over ADB, not a rebuild.

Component	Responsibility	Key APIs
Notification listener	Receives every posted notification; keeps only TikTok package + title/text match; drops group summaries; hands a `PublishJob` (key, postTime, contentIntent) to the engine; fires `contentIntent.send()` with the BAL opt-in; lists active notifications for the watchdog	`NotificationListenerService`, `StatusBarNotification`, `PendingIntent.send(ActivityOptions)`
Accessibility service	Exposes `snapshot()` (current package + flat list of nodes with text, content-desc, view id, clickable, enabled, bounds) and `click(node)`, `tap(x,y)`, `back()`, `home()`, `openShade()`, `lockScreen()`, `screenshot()`; restricted to TikTok and SystemUI packages	`AccessibilityService`, `AccessibilityNodeInfo`, `dispatchGesture`, `performGlobalAction`, `takeScreenshot`
Flow engine	Runs any registered Flow (trigger matcher + state machine); v1 registers one, section 6; single `Mutex` so only one run executes; polls `snapshot()` every 500 ms until a screen matches or a timeout fires; records every step	Kotlin coroutines on a single-thread dispatcher inside the accessibility service process

Component	Responsibility	Key APIs
Gate + UI	Master switch persisted in DataStore; Quick Settings tile mirrors it; main screen shows service status, last 20 runs, settings (quiet hours, auto vs confirm mode, dry-run) and a "dump current screen" debug button	`TileService`, `DataStore`, Compose
Device controller	Turns the display on, dismisses a swipe keyguard, remembers whether the phone was locked, relocks on completion	Transparent activity with `setTurnScreenOn` / `setShowWhenLocked`, `KeyguardManager.requestDismissKeyguard`, `GLOBAL_ACTION_LOCK_SCREEN`
Run log + watchdog	Room tables `runs` and `steps`; failure screenshot saved to app storage; periodic worker every 15 min re-scans active notifications for an unhandled match, checks both services are enabled, prunes logs older than 30 days; posts the bot's own success/failure notifications	`Room`, `WorkManager`, `NotificationManager`
Keep-alive (optional)	Foreground service of type `specialUse` with a quiet persistent notification, enabled only if the OEM kills the process	`ForegroundService`

Process model: both services are bound by the system and restarted if the process dies, so the app needs no scheduler of its own. The engine runs inside the accessibility service's process because that is the process holding the BAL exemption and the screen access.

Flow state machine



flow state machine: 8 states, 4 timeout exits

The engine never taps blind: each state polls the screen every 500 ms for one expected condition and either advances or fails on its timeout. Failure means screenshot, log, alert, and leave TikTok as it is; the one automatic retry happens only when TikTok never opened (10 minutes later, once).

State	Condition to advance	Action	Timeout	On failure
Triggered	Notification: package in {`com.zhiliaoapp.musically`, `com.ss.android.ugc.trill`}; title matches `^Your content from OnePublish is ready`; text matches `^Edit your video before sharing`; not a group summary; not already handled (key + postTime)	Gate on → enqueue. Gate off → log "ignored" only	—	—
Prepare device	Display interactive and keyguard not showing	Wake display; `requestDismissKeyguard` if keyguard is non-secure; remember prior lock state	5 s	Fail "locked" (phone has a secure lock and is locked)
Open draft	Foreground package == TikTok (window-state event)	`contentIntent.send()` with BAL opt-in. If no TikTok window in 8 s: `openShade()`, find SystemUI node whose text contains "OnePublish is ready", click it, wait 8 s more	8 s + 8 s	Fail "open" → eligible for the single delayed retry

State	Condition to advance	Action	Timeout	On failure
Editor	A clickable, enabled node with text "Next" exists in TikTok	Dismiss known dialogs first (section 8), then click Next	30 s	Fail "editor"
Post page	A clickable, enabled node with text "Post" exists; a node with text "Next" no longer exists	Click Post	30 s	Fail "post-page"
Confirm	Post page gone (no "Post" node) and TikTok still foreground, or an upload/progress indicator appears	None; record the timestamp	60 s	Fail "confirm" (Post was tapped: no retry; human checks TikTok)
Done	—	`home()`; `lockScreen()` if the phone was locked at trigger time; dismiss the TikTok notification; post a success notification	—	—

Selector order for every button: exact text match first, then content-description, then a view-id listed in `selectors.json`, then coordinates captured in the spike as a last resort. Any match must also be `isClickable && isEnabled` with non-empty bounds. If a text node is not itself clickable, the engine clicks its nearest clickable ancestor.

Hard rules the engine enforces regardless of config:

- Only one run at a time; new notifications queue behind it.
- Clicks are only performed when the foreground package is TikTok or SystemUI (shade fallback).
- Never click a node whose text matches Delete, Discard, Log out, Remove, Report, Not now (except in the listed dialog table), or anything with a currency symbol.
- Never type text in v1.
- At most 3 dialog dismissals per run; a fourth unexpected screen is a failure.
- A whole run is capped at 120 s wall-clock.
- Quiet hours (configurable, default none) delay runs rather than drop them.

Prerequisites and phone conditions

The bot works on a phone set up once as below. The app's main screen checks the items marked "checked by app" and shows a red row for any that fail.

Area	Requirement	Checked by app	Notes
Install	APK installed via `adb install` or Run from Android Studio	—	Avoids the Android 13+ restricted-settings block. Keep the signing key; a reinstall with the same key keeps granted services
Permissions	Notification access granted; Accessibility service enabled; POST_NOTIFICATIONS granted; battery optimization exemption granted	yes	Each has a deep link button in the app
OEM battery	Autostart / background activity allowed for the bot app (Xiaomi, Oppo, Vivo, Realme, Infinix, Tecno, Samsung "never sleeping apps")	partly	dontkillmyapp.com lists the per-brand steps. Tell me the phone brand and I will put the exact steps here
Lock screen	Either no secure lock (swipe), or Extend Unlock keeping the phone unlocked where it sits (trusted place or Bluetooth device), or the phone is simply left unlocked	yes (reports lock state at trigger time)	With a PIN and a locked phone the run fails at "Prepare device". PIN entry by the bot is deliberately out of scope
Display	Any state; the bot turns the screen on. Portrait orientation. Font size and display size at default or close to it	—	Text selectors do not depend on layout, coordinate fallbacks do
Network	Online at trigger time and for the minute after	yes (warns if offline when triggered)	TikTok's upload runs from TikTok, not from the bot

Area	Requirement	Checked by app	Notes
TikTok app	Installed in the main profile (not Work profile, Private Space or Secure Folder); logged into the account OnePublish authorized; app language English; version ≥ 31.8 and reasonably current; notifications enabled for TikTok, including the category the inbox notification uses	package + version yes; login/language no	A different account or language fails at "Editor" with a screenshot
Android	Android 15 (API 35). Advanced Protection Mode off after any later upgrade to Android 16/17	yes	See section 4
Do Not Disturb	Any	—	DND suppresses sound, not delivery to the listener
Phone in use	Preferably idle. In "auto" mode the bot takes over the screen for about 20 s whenever the notification lands	—	"Confirm" mode (section 12, open question 3) shows a bot notification first and runs only after you tap Run, or automatically after N minutes
OnePublish	Caption/title set there; no more than 5 unposted drafts in 24 h	—	TikTok's pending-share cap, section 3

One-time setup after install takes about five minutes: grant the four permissions, add the Quick Settings tile, run a dry run, then a real post.

Edge cases and failure modes

Every case below ends in one of three outcomes: handled automatically, failed safely with a screenshot and an alert, or recovered by the 15-minute watchdog. None of them can post the wrong thing, because the bot only ever taps Next and Post on a screen TikTok opened from its own notification.

#	Case	Likelihood	Handling
1	Notification arrives while the bot process is dead or the phone is rebooting	Medium	Listener re-binds on restart and receives the still-active notification; watchdog also re-scans `getActiveNotifications()` and triggers any unhandled match
2	Notification was swiped away or TikTok auto-cancelled it before the run	Low	Nothing to click. v1 fails "open" and alerts; v1.1 adds the in-app fallback: open TikTok → Inbox → System notifications → tap the item
3	Two or more drafts arrive together (OnePublish batch)	Medium	Jobs queue; runs are sequential; each notification keyed by `key + postTime` so none is skipped or double-run
4	Grouped TikTok notifications (summary + children)	High	Summary (`FLAG_GROUP_SUMMARY`) ignored; each child evaluated on its own
5	`contentIntent.send()` is blocked by BAL on this OEM build	Low (service is exempt)	Detected by "no TikTok window in 8 s"; shade fallback taps the notification via SystemUI, which always has launch rights
6	Notification tap lands on TikTok's inbox list instead of the editor	Low-medium (TikTok decides)	Editor state also accepts an inbox item whose text contains "OnePublish" and clicks it once, then waits for Next again
7	Phone locked with a PIN	Depends on setup	Fail "locked"; alert; notification stays for a manual tap. Prerequisite table says how to avoid
8	Screen off, no lock	Common	Device controller turns the display on; run proceeds; screen turned back off/locked afterwards only if it was locked before

#	Case	Likelihood	Handling
9	User is actively using the phone	Medium	Auto mode: run anyway (20 s takeover). Confirm mode: bot notification with Run now; auto after N min
10	TikTok shows a dialog before the editor (what's new, permission, promo, "Continue editing?")	Medium	Dialog table in `selectors.json`: dismiss buttons in order of preference — Not now, Later, Skip, Close, OK, Back. Max 3 per run
11	Next is greyed out while the video is still processing	Medium	Wait for `isEnabled` with the 30 s timeout; the spike measures typical processing time
12	Post page shows a mandatory choice (AI-generated label, content disclosure, audience)	Medium	Known prompts added to the dialog table with the *decline/skip* answer; an unknown mandatory prompt fails "post-page" with a screenshot so you can add it
13	TikTok asks to confirm posting without caption or sound	Low	Config says OnePublish pre-fills the caption; if the prompt appears, treat as dialog with "Post anyway"-type button only if its text is listed
14	Post tapped but upload fails on TikTok's side	Low	Confirm state sees a failure text or nothing within 60 s → Fail "confirm"; no retry, because retrying could double-post
15	TikTok logged out, wrong account, or update required	Low	Unexpected screen → Fail "editor" with screenshot
16	TikTok app language not English or phone locale changes button text	Low	Selectors fail → Fail with screenshot; config supports alternate strings
17	TikTok UI update renames or restyles Next/Post	Certain, over months	Fails safely; `dump_screen.sh` (spike) shows the new node; edit `selectors.json`; no rebuild
18	View tree empty on a TikTok screen (custom rendering)	Low on the post flow	Coordinate fallback captured in the spike; screenshot attached to the run for diagnosis

#	Case	Likelihood	Handling
19	Accessibility service gets disabled (OEM reset, crash loop, user toggled)	Medium on some OEMs	Watchdog checks `AccessibilityManager` every 15 min and posts a persistent warning with a deep link to the setting
20	Notification access revoked	Low	Same watchdog check
21	Process killed mid-run	Low	Run marked "interrupted" on next start; the still-active notification is handled as case 1; if Post was already tapped the log says so and no retry happens
22	Another TikTok notification with similar wording (e.g. a download ready)	Low	Full title regex plus text regex plus package; anything else is ignored and logged
23	More than 5 unposted drafts because the bot was off	Possible	OnePublish-side cap; bot shows a count of unhandled matches so you notice
24	Night-time run turns the screen on in a dark room	Common	Quiet hours option delays runs to a window you set
25	Bot taps Post, then TikTok shows "Add to playlist" / share sheet	Medium	Confirm state treats those as success; Done presses Home which closes them
26	Play Protect flags the sideloaded app	Low	Allow it in Play Protect; it is your own signed debug/release build

Risks, restrictions and boundaries

The real risks are maintenance and device setup, not feasibility. Ranked by expected pain:

Risk	Impact	Chance	Mitigation
TikTok changes its post flow (button text, extra steps, new mandatory prompts)	Flow fails until config is updated	High over a year	Text selectors in config; failure screenshots; dump script; the bot never guesses so a change costs a missed post, not a wrong post

Risk	Impact	Chance	Mitigation
OEM battery manager kills the process or silently disables the accessibility service	Missed notifications	Medium, brand-dependent	Exemptions + autostart; optional foreground service; watchdog alerts within 15 min
TikTok stops sending a push and keeps the item only in the in-app inbox	Trigger never fires	Low	v1.1 in-app inbox fallback (edge case 2), driven by the watchdog or a schedule
Android upgrade enables Advanced Protection Mode or tightens accessibility further	Service blocked	Low on this phone; rising with Android 17	Keep the mode off; declare `isAccessibilityTool` if needed; this is a personal build so Play policy does not apply
TikTok detects automation and flags the account	Account restriction	Low: the bot performs two human-paced taps on TikTok's own flow, a few times a day, from the account's usual phone	Human-like pacing (0.8–2 s between taps); no bulk actions; cap runs per hour in config
The bot taps Post on a draft you did not want live	Unwanted public post	Low; the trigger is specific and the audience comes from TikTok's settings	Confirm mode for a trial period; dry-run; per-run log with screenshot of the post page before Post is tapped
Accessibility service exposure: the app can read any screen it is scoped to	Privacy, if the phone is shared or the APK leaks	Low	Scope `packageNames` to TikTok and SystemUI only; no network permission; logs stay on-device; keep the repo private
Phone left unlocked for the bot	Physical security	Your call	Extend Unlock with a trusted place is the compromise

Hard boundaries of this design, so you are not surprised later:

- It is reactive. No notification, no run. There is no polling of TikTok's inbox in v1.
- It cannot act on the lock screen with a secure lock.
- It cannot run inside a Work profile or Private Space.
- It depends on TikTok's own tap intent and UI; TikTok can change both without notice.
- It is single-device and single-account by design.
- It does not survive a factory reset or reinstall without re-granting both services.

Verification spike (before writing the app)

One to two hours with the phone on USB settle every open question in this document. Nothing here needs code beyond ADB; the outputs feed `selectors.json` directly.

- [ ] Confirm the TikTok package: `adb shell pm list packages | grep -iE 'musically|trill'` and `adb shell dumpsys package <pkg> | grep versionName`
- [ ] Trigger a real OnePublish upload and capture the notification: `adb shell dumpsys notification --noredact | grep -A 30 <pkg>` → record exact title, text, channel id, whether `contentIntent` is an activity, and the group key
- [ ] Tap the notification by hand and dump each screen with `adb shell uiautomator dump /sdcard/ui.xml && adb pull /sdcard/ui.xml`: the editor (find the Next node: text, content-desc, resource-id, clickable, enabled, bounds), the post page (same for Post), and any dialog that appears
- [ ] Note how long Next stays disabled after the editor opens (sets the 30 s timeout or shortens it)
- [ ] Repeat the dump with the phone in landscape and with a larger font size to confirm text selectors still match
- [ ] Send the notification's intent from the shell while TikTok is in the background and the screen is on: `adb shell cmd notification` has no send; instead install the 20-line test APK (notification listener only, sends `contentIntent` with the BAL opt-in) and check `adb logcat | grep -i 'background activity'` — a "Background activity launch blocked" line means the fallback path is the primary path on this OEM
- [ ] With the screen off and the lock set to swipe, run the same test APK and confirm the display wakes and TikTok appears
- [ ] Check which lock the phone uses today and whether Extend Unlock is available in Settings
- [ ] Record the OEM and Android build (`adb shell getprop ro.product.manufacturer ro.build.version.release`) so the battery-exemption steps can be written for it

- [ ] Watch for post-Post screens (share sheet, playlist prompt, upload progress) and capture one dump of each

Stop criteria: if the Next or Post node has no text, no content-desc and no stable resource-id, the plan switches to coordinate taps captured here plus a screenshot-based sanity check, and the risk table's maintenance line moves to "very high". Everything else in this document stays the same.

Requirements

Functional requirements are numbered FR, non-functional NFR. "Must" items define v1 done.

ID	Requirement	Priority
FR-1	Detect a TikTok notification matching the OnePublish title and text regexes within 2 s of posting	Must
FR-2	Open the draft via the notification's own tap intent; fall back to tapping it in the shade	Must
FR-3	Tap Next on the editor and Post on the post page using text selectors, with the waits and timeouts of section 6	Must
FR-4	Confirm the upload started; press Home; relock if the phone was locked; dismiss the handled notification	Must
FR-5	Master switch persisted across reboots; Quick Settings tile and in-app switch stay in sync; Off ignores notifications and logs them	Must
FR-6	Run log: one row per run (trigger time, outcome, duration, failure reason) and one row per step; failure screenshot stored and viewable	Must
FR-7	Bot notifications: success (dismissable), failure (with screenshot thumbnail and Open TikTok action), service-disabled warning (persistent)	Must
FR-8	Watchdog every 15 min: service status, unhandled matching notifications, log pruning	Must

ID	Requirement	Priority
FR-9	Selectors, regexes, dialog table and timeouts loaded from `selectors.json` in app files; a built-in default is used when the file is absent	Must
FR-10	Dry-run mode: full flow, stops before tapping Post, logs what it would tap	Must
FR-11	Setup screen with status rows and deep links for each permission	Must
FR-12	Confirm mode: bot notification with Run now / Skip, auto-run after N minutes	Should
FR-13	Quiet hours window	Should
FR-14	Debug: dump current foreground screen tree to a file	Should
FR-15	In-app inbox fallback when the push is missing	Later (v1.1)
NFR-1	Kotlin, Compose, minSdk 35, single module, no third-party SDKs beyond AndroidX, Room, WorkManager	Must
NFR-2	No INTERNET permission; nothing leaves the device	Must
NFR-3	Accessibility service scoped to TikTok and SystemUI packages; no screen content logged except node text of the TikTok post flow	Must
NFR-4	A run completes in under 60 s in the normal case and is hard-capped at 120 s	Must
NFR-5	Idle cost: no wake locks, no polling; CPU wake only on events and the 15-min worker	Must
NFR-6	Crash-free: every tap wrapped in try/catch; an exception ends the run as Failed, never the service	Must
NFR-7	Reinstall with the same key keeps settings and granted services	Should
NFR-8	Code structured so a second flow is a new `Flow` class plus a matcher, not a change to the engine	Should

Build plan, test plan and open questions

Five milestones, each ending in something you can run on the phone. With Claude Code doing the Kotlin, milestones 1–3 are a weekend; 4–5 are a week of normal use.

1. **Spike** (section 10). Output: `selectors.json` v1, notification sample, OEM notes, BAL result. Gate: Next and Post identified by text or id.
2. **Skeleton.** Project, manifest, both services registered, setup screen with permission status, master switch + tile, Room log. Gate: both services show enabled; a fake notification from a test app is logged as ignored/matched.
3. **Flow.** Engine + TikTok flow in dry-run only. Gate: ten dry runs on real OnePublish drafts reach the post page and stop, with correct logs.
4. **Live.** Enable Post; confirm mode on; watchdog; failure screenshots; bot notifications. Gate: five real posts in a row published with no manual touch; one induced failure (rename a selector) produces a screenshot and alert.
5. **Harden.** Auto mode, quiet hours, OEM battery steps, 7-day soak. Gate: a week with zero missed drafts, or every miss explained by the log.

Test plan (run at milestone 4 and after every TikTok update):

Test	Expected
Normal: screen off, swipe lock, phone idle	Post published within 60 s; screen off again; success notification
Screen on, user mid-task (auto mode)	Bot takes over and finishes; control returned to the previous app via Home
Confirm mode	Nothing happens until Run now; auto-run after the configured delay
Gate off	Notification logged as ignored; nothing opens
Two drafts back to back	Both posted, sequentially, no double tap
Process killed before the run (`adb shell am force-stop <bot>`) then notification arrives	Services re-bind; run happens or watchdog catches it within 15 min
Airplane mode at trigger	Run fails at Confirm or Open with a clear reason; alert
Selector renamed in config	Fail at the right state with a screenshot; no tap on anything else

Test	Expected
TikTok force-stopped before trigger	Intent still opens TikTok cold; editor appears (longer wait)
Reboot	Switch state restored; services enabled; watchdog scheduled

Open questions for you (answers change small parts of the design, not the verdict):

- [ ] Phone brand and model, for the exact battery/autostart steps
- [ ] Lock screen today: swipe, PIN, fingerprint? Willing to use Extend Unlock at home?
- [ ] Auto or confirm mode for the first week?
- [ ] Does OnePublish let you set the caption before upload, or do you want the bot to type one later (v1.1)?
- [ ] Should the bot post during the night, or hold until a quiet-hours window ends?
- [ ] Does your phone run TikTok as `com.zhiliaoapp.musically` or `com.ss.android.ugc.trill`? (spike item 1)

Sources used in this document: TikTok for Developers (Content Posting API, Upload, Get post status, Content sharing guidelines, legacy Share Video); Android Developers (Background activity starts, Secure BAL); XDA on Android 13 restricted settings; Android Authority on Android 15 sensitive notifications and Android 17 Advanced Protection; Postiz TikTok notes.